

Water Hero - Complete Developer & Logic Specification Document

Project Name: Water Hero Application

Document Purpose: Technical, Mathematical, and Logic Implementation Blueprint for Software Engineers.

Developer/Author: Mohamed Mohamed Rashad

Language Standard: English (Technical Standard)

1. Overview & System Context

This document serves as an exhaustive technical specification for developers building or refactoring the **Water Hero** web and mobile application. The application tracks domestic water consumption across user activities, computes instantaneous and projected financial impacts, evaluates activity duration against recommended ecological limits, and displays actionable savings insights.

2. Global Constants & Configuration Data

The application relies on global configurable parameters stored in the application context state. Developers must maintain these values in a centralized configuration store or environment context.

Constant Identifier	Data Type	Value	Unit	Technical Description
app_title	String	"Water Hero"	N/A	The display title of the application shown in headers and app metadata.
water_price_per_liter	Float	0.008	EGP / Liter	The standard tariff multiplier for calculating

Constant Identifier	Data Type	Value	Unit	Technical Description
				water utility costs per liter consumed.
developer_name	String	"Mohamed Mohamed Rashad"	N/A	Author attribution displayed in the About screen.
DAYS_IN_WEEK	Integer	7	Days	Multiplier for weekly financial and volume projections.
DAYS_IN_MONTH	Integer	30	Days	Multiplier for monthly financial and volume projections.
DAYS_IN_YEAR	Integer	365	Days	Multiplier for annual projection calculations.

Developer Implementation Guide: Ensure `water_price_per_liter` can be dynamically overridden via application configuration (e.g., `elementSdk` or user settings). All financial calculations across components MUST reference this centralized constant directly rather than hardcoding local numbers.

3. Activity Catalog & Consumption Rates

Each tracked activity is defined by a data schema containing its unique key, UI iconography, base water flow rate (Liters per minute), and acceptable duration boundaries (min and max duration thresholds).

Activity Key	UI Display Label	Icon Tag	Base Water Flow Rate (L/min)	Min Threshold (min)	Max Threshold (min)
Showering	Showering	🚿	15.0	3.0	15.0
Dishwashing	Dishwashing	🍽️	12.0	3.0	10.0
PlantWatering	Plant Watering	🌿	8.0	2.0	10.0
CarWashing	Car Washing	🚗	20.0	10.0	30.0
Laundry	Laundry	👕	18.0	20.0	60.0
Cleaning	Cleaning	🧹	10.0	5.0	20.0
HandWashing	Hand Washing	🧼	3.0	0.5	2.0
Cooking	Cooking	🍳	6.0	2.0	8.0

Developer Implementation Guide: Store this catalog as a key-value object or hash map for O(1) time complexity access during dynamic lookup operations in the calculator and activity logger UI modules.

4. Core Mathematical Logic & Calculations

4.1 Single Event Consumption & Cost Formulas

When a user inputs or simulates a specific activity duration, the system computes total water wastage and financial cost using the formulas below:

```
// 1. Water Wastage (Liters) Wastage_Liters = Activity_Base_Rate * Duration_Minutes // 2. Financial Cost (EGP) Event_Cost = Wastage_Liters * Water_Price_Per_Liter
```

Technical Explanation for Developers:

- Activity_Base_Rate is retrieved from the Activity Catalog based on selected activity key.
- Duration_Minutes is a positive floating-point or integer value supplied by the user UI slider or numeric input field.
- Wastage_Liters represents the total volume of water consumed in liters. Round output using Math.round() when rendering UI volume labels.
- Event_Cost is rounded to two decimal places (e.g., .toFixed(2)) for currency display in EGP.

4.2 Time Projections (Weekly, Monthly, Yearly)

To provide predictive insights, single-day consumption figures are extrapolated across temporal boundaries:

```
// Extrapolations based on Daily_Wastage and Daily_Cost
Weekly_Water_Liters = Daily_Wastage_Liters * 7
Weekly_Cost_EGP = Daily_Cost_EGP * 7
Monthly_Water_Liters = Daily_Wastage_Liters * 30
Monthly_Cost_EGP = Daily_Cost_EGP * 30
Yearly_Water_Liters = Daily_Wastage_Liters * 365
Yearly_Cost_EGP = Daily_Cost_EGP * 365
```

Technical Explanation for Developers: Use these formulas in the interactive **Simulator / Calculator Module** when users adjust the input slider. Compute recalculations synchronously on input event fires.

5. Conditional Business Logic & Rule Engine

5.1 Duration Appropriateness Validator

Whenever an activity log is created, the system compares the elapsed time against acceptable minimum and maximum bounds defined for that activity.

```
function checkTimeAppropriate(activityKey, durationMinutes) { const activity = ActivityCatalog[activityKey]; if (!activity) return false; return (durationMinutes >= activity.minTime) && (durationMinutes <= activity.maxTime); }
```

UI Trigger & Modal Outcome:

- **If true:** Display Modal with Success Icon (✅), Green Border Style, and Title: *"Perfect Duration!"* 🎉
- **If false:** Display Modal with Warning Icon (⚠️), Red Border Style, and Title: *"Too Long Duration!"* 🕒 advising user to reduce time.

5.2 Consumption Impact Classifier

In the simulator/calculator module, dynamic comparison messages are rendered based on calculated daily water volume thresholds:

Daily Volume Condition (Liters)	Classification Category	Rendered Alert Message String
Wastage <= 20	Optimal / Minimal	"💧 Water consumption is low & efficient!"
21 <= Wastage <= 50	Moderate	"🌊 Moderate consumption level."
51 <= Wastage <= 100	High Warning	"⚠️ High consumption! Try reducing duration."
Wastage > 100	Critical Excessive	"🔴 Very high consumption! Urgent reduction needed."

Developer Implementation Guide: Implement this via a chained if-else or standard switch range function. Ensure text dynamically updates as the UI slider moves.

6. Data Models & Local Storage Persistence

6.1 Record Data Schema

Activity entries captured by the user must be structured according to the JSON object schema below before persisting to localStorage:

```
{ "id": "String (Timestamp e.g. Date.now().toString())", "date": "String (ISO Date Format YYYY-MM-DD)",
  "activity": "String (Activity Display Name / Key)", "duration": "Number (Duration in Minutes)", "wastageAmount":
  "Number (Calculated Volume in Liters)", "cost": "Number (Calculated Cost in EGP)", "timeAppropriate": "Boolean
  (Validation Result)" }
```

6.2 Persistence Contract

- **Storage Key:** "waterHeroRecords"
- **Read Operation:** On application initial boot, parse JSON from localStorage.getItem('waterHeroRecords'). Fallback to empty array [] if null or invalid.
- **Write Operation:** On activity submission, push the validated record object into state array and invoke localStorage.setItem('waterHeroRecords', JSON.stringify(allRecords)).

7. Tips & Conservation Knowledge Base Logic

The Tips tab presents pre-calculated impact benchmarks comparing wasteful habits vs. recommended habits. Developers can use this static reference table for rendering advice cards:

Action / Tip Title	Standard vs Recommended Practice	Daily Water Saved (L)	Daily Financial Saved (EGP)
Shorten Shower Time	Reduce shower duration from 15 mins to 5 mins	150.0 L	1.20 EGP
Bucket Car Washing	Use bucket (10 mins) instead of hose (30 mins)	400.0 L	3.20 EGP
Sink Dishwashing	Use filled sink basin vs continuous running tap	84.0 L	0.67 EGP
Smart Plant Watering	Reuse leftover drinking/grey water for garden	80.0 L	0.64 EGP
Tap off Hand Washing	Turn off tap while applying soap (30s vs 2m)	4.5 L	0.036 EGP
Cumulative Best-Practice Total	Applying all 5 conservation tips daily	718.5 L / day	5.748 EGP / day (172.44 EGP / month)

8. Application Architecture & Tab Navigation State

The application single-page interface (SPA) manages view transitions using simple DOM visibility toggles across 5 key views:

1. content-home: Displays today's total wastage, cost metrics (daily, weekly, monthly), top activity bar charts, and quick statistics.
2. content-calculator: Interactive simulation view with activity selector dropdown and duration slider.
3. content-add: Action logging view with interactive activity selection buttons and duration submission inputs.
4. content-tips: Static/dynamic conservation tips list detailing water/money savings potential.
5. content-about: Developer attribution, app purpose statement, and feature list.

Developer Implementation Guide: Ensure view changes strip/add the hidden CSS utility class on container divs and update active tab button highlight borders accordingly.